

[컴퓨터프로그래밍3] Term project 보고서

컴퓨터융합학부 202202598 백정은

1. 프로젝트 개요

이번 프로젝트에서는 C 언어로 학생 정보를 관리하는 shell 프로그램을 구현했다. 프로그램은 학생 정보를 linked list로 메모리에 저장하고, CSV 파일을 통해 데이터를 영구 저장한다. 사용자는 터미널에서 명령어를 입력해 학생 추가, 삭제, 수정, 검색, 출력 등의 기능을 사용할 수 있다.

프로그램은 Admin Program과 Client Program 두 가지 실행 파일로 제공되며, 같은 소스 코드를 기반으로 preprocessing flag를 이용해 각각 빌드한다.

2. 프로그램 구조

파일은 기능별로 분리했다.

파일	역할
main.c	프로그램 시작, 인자 처리, 셸 루프
student.h / student.c	Student 구조체 정의, linked list 함수 구현
file_io.h / file_io.c	CSV 파일 읽기/쓰기
command.h / command.c	명령어 파싱, command table, 핸들러 구현
Makefile	빌드 자동화

프로그램 실행 흐름은 다음과 같다.

```
프로그램 시작
→ CSV 파일 로드 (load_csv)
→ 셸 루프 또는 명령어 파일 실행
→ 입력 파싱 (dispatch_command)
→ command table에서 명령어 찾기
→ 핸들러 함수 호출
→ exit 시 메모리 해제 후 종료
```

3. 자료구조 설계

학생 정보는 아래 구조체로 표현하고, 단방향 linked list로 관리한다.

```
typedef struct Student {
    int id;
    char name[32];
    int score;
    struct Student *next;
} Student;
```

주요 함수:

- add_student() : 리스트 끝에 노드 추가
- find_student() : id로 노드 탐색

- `delete_student()` : id로 노드 삭제 후 메모리 해제
- `free_list()` : 전체 노드 메모리 해제

삽입은 리스트 끝에 추가하므로 입력 순서가 유지된다. 삭제는 이전 노드의 `next` 포인터를 수정하는 방식으로 구현했으며, head 노드 삭제를 위해 이중 포인터(`Student head`)를 사용했다.

4. CSV 파일 형식

```
id,name,score
1,Alice,90
2,Bob,85
3,Charlie,95
```

- 첫 번째 줄은 반드시 `id,name,score` 헤더여야 한다. 헤더가 다르면 오류를 출력하고 종료한다.
- `id` 는 양의 정수, 고유값이어야 한다.
- `name` 은 빈 문자열이거나 심표를 포함할 수 없다.
- `score` 는 0 이상 100 이하의 정수여야 한다.

CSV를 읽을 때는 `fgets()` 로 한 줄씩 읽고, `strtok()` 으로 심표 기준으로 파싱한다. `fgets()` 가 개행 문자를 포함해서 읽기 때문에 `strcspn()` 으로 `\n` 과 `\r` 을 제거하도록 했다.

저장할 때는 `fopen("w")` 으로 파일을 열고 헤더를 먼저 쓴 뒤 학생 정보를 순서대로 쓴다.

5. 명령어 명세

명령어 처리는 함수 포인터 기반 command table 구조를 사용했다.

```
typedef ShellResult (*CommandHandler)(char *args, Student **head);

typedef struct {
    const char *name;
    CommandHandler handler;
    const char *usage;
    const char *description;
} Command;
```

`dispatch_command()` 에서 입력을 파싱해 command table을 순회하며 이름이 일치하는 핸들러를 호출한다.

명령어	Admin	Client	설명	주요 오류
<code>list</code>	O	O	전체 학생 출력	학생 없으면 <code>No students found.</code>
<code>find <id></code>	O	O	ID로 학생 검색	없는 학생: <code>Error: student not found.</code>
<code>stats</code>	O	O	통계 출력	학생 없으면 <code>No student data available.</code>
<code>reload</code>	O	O	CSV에서 다시 불러오기	파일 오류 시 오류 메시지 출력
<code>help</code>	O	O	도움말 출력	-
<code>clear</code>	O	O	화면 지우기	-
<code>exit</code>	O	O	프로그램 종료	-

<code>save</code>	O	X	CSV에 저장	파일 쓰기 실패 시 오류 메시지 출력
<code>add <id> <name> <score></code>	O	X	학생 추가	중복 id, 비숫자 id/score, score 범위 오류, 빈 name
<code>delete <id></code>	O	X	학생 삭제	없는 학생: <code>Error: student not found.</code>
<code>update <id> <score></code>	O	X	점수 수정	없는 학생, score 범위 오류
<code>sort <name score></code>	O	X	정렬	잘못된 키: <code>Error: invalid sort key.</code>

6. Admin/Client Program 설계

두 프로그램은 동일한 소스 코드를 사용하며, 컴파일 시 `-DADMIN_MODE` 또는 `-DCLIENT_MODE` 플래그로 분기한다.

```
admin:
    $(CC) $(CFLAGS) -DADMIN_MODE $(SRCS) -o admin_shell

client:
    $(CC) $(CFLAGS) -DCLIENT_MODE $(SRCS) -o client_shell
```

command table을 `#ifdef` 로 분리해서 Admin은 모든 명령어를, Client는 조회 명령어만 포함하도록 했다.

```
#ifdef ADMIN_MODE
Command commands[] = {
    {"list", handle_list, ...},
    {"find", handle_find, ...},
    {"add", handle_add, ...},
    // ...
};
#endif

#ifdef CLIENT_MODE
Command commands[] = {
    {"list", handle_list, ...},
    {"find", handle_find, ...},
    {"reload", handle_reload, ...},
    // ...
};
#endif
```

Client에서 `add`, `delete`, `update`, `save` 등을 입력하면 command table에 없기 때문에 `Unknown command or permission denied.`를 출력한다.

`help` 명령어도 `#ifdef` 로 분기해서 Client에서는 사용 가능한 명령어만 출력한다.

7. 예외 처리 설계

프로그램이 비정상 종료되지 않도록 아래 상황들을 처리했다.

입력 오류

- 알 수 없는 명령어 → `Unknown command or permission denied.`

- 인자 부족 → `Error: missing argument.`
- 비슷자 id/score → `atoi()` 대신 `strtol()` 을 사용해 검증
- id가 0이거나 음수 → `Error: invalid id.`
- score가 0~100 범위 밖 → `Error: invalid score.`
- name이 빈 문자열이거나 심표 포함 → `Error: invalid name.`
- 중복 id → `Error: duplicate ID.`
- 존재하지 않는 학생 → `Error: student not found.`

파일 오류

- CSV 파일 열기 실패 → `Error: cannot open file`
- 잘못된 CSV 헤더 → `Error: invalid CSV header.`
- 명령어 파일 열기 실패 → 오류 메시지 출력 후 종료

명령어 파일 실행 중 오류

명령어 파일 실행 중 오류가 발생하면 해당 줄을 건너뛰고 다음 명령어를 계속 실행한다.

```
[command file:2] update 99 70
Error: student not found.
Skipped line 2.
[command file:3] find 1
ID: 1
Name: Alice
Score: 90
```

메모리 관리

`exit` 명령어 실행 시 셸 루프가 종료되고, `main()` 에서 `free_list()` 를 호출해 모든 노드를 해제한다.

8. 테스트 케이스와 결과

테스트 1: 기본 CRUD

```
$ ./admin_shell students.csv
[Admin Program]
Loaded 3 students from students.csv.
admin> list
ID   Name      Score
1    Alice      90
2    Bob        85
3    Charlie    95
admin> add 4 David 88
Student added.
admin> update 2 95
Student updated.
admin> delete 3
Student deleted.
admin> list
ID   Name      Score
1    Alice      90
2    Bob        95
4    David      88
admin> save
Saved 3 students to students.csv.
admin> exit
Goodbye.
```

테스트 2: 오류 입력 처리

```
admin> add abc David 88
Error: invalid id.
admin> add 5 David abc
Error: invalid score.
admin> add 1 Alice 90
Error: duplicate ID.
admin> find 999
Error: student not found.
admin> unknowncmd
Unknown command or permission denied.
```

테스트 3: 명령어 파일 실행

```
$ ./admin_shell students.csv -f commands.txt
[Admin Program]
Loaded 3 students from students.csv.
[command file:1] list
ID   Name    Score
1    Alice   90
2    Bob     85
3    Charlie 95
[command file:2] add 4 David 88
Student added.
[command file:3] update 99 70
Error: student not found.
Skipped line 3.
[command file:4] find 4
ID: 4
Name: David
Score: 88
[command file:5] save
Saved 4 students to students.csv.
[command file:6] exit
Goodbye.
```

테스트 4: Client 권한 제한

```
$ ./client_shell students.csv
[Client Program]
Loaded 3 students from students.csv.
client> add 4 David 88
Unknown command or permission denied.
client> list
ID   Name    Score
1    Alice   90
2    Bob     85
3    Charlie 95
client> exit
Goodbye.
```

테스트 5: stats

```
admin> stats
Count: 3
Average: 90.0
Max: 95
Min: 85
```

9. 한계점과 개선 방향

- **id 순서 보장 없음:** 학생을 추가할 때 리스트 끝에 붙이기 때문에 id 순서와 저장 순서가 다를 수 있다. 삽입 정렬 방식으로 개선 가능하다.
- **대용량 파일 성능:** linked list 탐색이 $O(n)$ 이라 학생 수가 많아지면 느려질 수 있다. 해시 테이블 등을 사용하면 개선 가능하다.